

Embedded Systems & IoT

Introduction & Mini Project Report

Internship Task-1 Documentation

Table of Contents

1. Introduction to Embedded Systems
2. Component Study
3. Mini Project: Smart Temperature Monitor
4. Implementation & Code
5. Real-World Applications
6. Conclusion

1. Introduction to Embedded Systems

1.1 What is an Embedded System?

An embedded system is a specialized computer system designed to perform dedicated functions within a larger mechanical or electrical system. Unlike general-purpose computers, embedded systems are optimized for specific tasks with constraints on power consumption, size, cost, and processing capability. They operate with minimal human intervention and are typically non-interactive.

Key Characteristics:

- Single-purpose: Designed to perform one specific function
- Real-time operation: Must respond within strict time constraints
- Low power consumption: Often battery-powered or energy-efficient
- Compact size: Integrated into larger systems
- Cost-effective: Mass production reduces per-unit cost

1.2 Real-World Applications

Smart Home Systems

Smart lighting (automatic brightness control), temperature management (smart thermostats), security systems (motion detection, door locks), and voice assistants (Amazon Alexa, Google Home) all utilize embedded systems to provide automated, connected home experiences.

Automotive Applications

Modern vehicles contain 50-100+ embedded systems managing engine control units (ECU), anti-lock braking systems (ABS), airbag deployment, infotainment systems, and autonomous driving features.

Healthcare Devices

Pacemakers, insulin pumps, glucose monitors, pulse oximeters, and ECG machines rely on embedded systems for critical patient monitoring and medication delivery with high reliability and long battery life.

Industrial Automation & Robotics

Manufacturing robots, PLCs (Programmable Logic Controllers), industrial sensors, and control systems use embedded systems for precision automation and real-time process control.

IoT Devices

Connected devices like smartwatches, fitness trackers, environmental sensors, and wireless gateways form the backbone of IoT ecosystems for data collection and remote monitoring.

1.3 Microcontroller vs Microprocessor

Feature	Microcontroller	Microprocessor
Purpose	Single task operation	Multitasking computing
Integrated Components	RAM, ROM, I/O on chip	CPU only (separate chips)
Power Consumption	Low (mA to μ A)	High (W)
Cost	Low	High
Applications	IoT, Embedded Systems	Computers, Servers

For embedded systems, microcontrollers are preferred due to their integrated peripherals, lower power consumption, cost-effectiveness, and suitability for single-task operations. Microprocessors are used in systems requiring high computational power and multitasking capabilities.

2. Component Study

2.1 Microcontroller Platforms

Arduino UNO

Arduino UNO is an open-source microcontroller board based on the ATmega328P. It features 14 digital I/O pins, 6 analog input pins, 16 MHz clock speed, and 32 KB flash memory. Ideal for beginners and hobbyists due to its large community support and extensive libraries. Operating voltage: 5V.

ESP32

ESP32 is a powerful dual-core microcontroller with built-in Wi-Fi and Bluetooth connectivity. It offers 240 MHz clock speed, 520 KB RAM, and extensive GPIO pins. Excellent for IoT projects requiring wireless communication. Operating voltage: 3.3V. Significantly more powerful than Arduino UNO.

Raspberry Pi Pico

Raspberry Pi Pico is a compact, low-cost microcontroller based on the RP2040 chip with dual-core ARM Cortex-M0+. Features 2 MB flash memory, 26 GPIO pins, and operates at 125 MHz. No built-in wireless, but excellent for real-time tasks and DSP applications. Operating voltage: 3.3V.

2.2 Sensors Used in Embedded Systems

Temperature Sensor (TMP36)

- Type: Capacitive humidity and thermistor temperature sensor
- Temperature Range: -40°C to +80°C (DHT22) or 0°C to 50°C (TMP36)

- Accuracy: $\pm 0.5^{\circ}\text{C}$ (DHT22) or $\pm 2^{\circ}\text{C}$ (TMP36)
- Output: Digital signal via single wire protocol

Motion Sensor (PIR - Passive Infrared)

- Type: Detects infrared radiation emitted by moving objects
- Output: Digital HIGH (1) when motion detected, LOW (0) when no motion
- Detection Range: 5-10 meters
- Response Time: 0.3-2 seconds (configurable)

Light Sensor (LDR - Light Dependent Resistor)

- Type: Photoresistor that changes resistance based on light intensity
- Resistance in Darkness: $1\text{ M}\Omega$ (1,000,000 Ω)
- Resistance in Bright Light: 10-20 k Ω
- Output: Analog voltage (requires ADC for microcontroller)

3. Mini Project: Smart Temperature Monitor

3.1 Project Overview

The Smart Temperature Monitor is an IoT-enabled device that continuously measures ambient temperature and displays real-time alerts when the temperature crosses user-defined thresholds. The system consists of a TMP36 analog temperature sensor, a 16x2 LCD display, LEDs for visual alerts, and a buzzer for audio notifications.

Project Objectives:

- Read analog data from TMP36 temperature sensor
- Display real-time temperature on LCD
- Trigger visual (LED) and audio (buzzer) alerts when thresholds are crossed
- Store and log temperature data for analysis

3.2 System Architecture

Hardware Components:

Component	Specification	Function
Arduino UNO	ATmega328P, 16MHz, 32KB Flash	Main controller and logic processor
TMP36	Temp: 0-50°C,	Temperature sensing
16x2 LCD	16x2 characters	Display sensor readings and alerts
Green LED	5V, 20mA, 2mm	Indicates normal operation
Red LED	5V, 20mA, 2mm	Visual alert for threshold crossing
Piezo Buzzer	5V, 85dB @ 10cm	Audio notification for alerts
Resistors	220 Ω (LED), 4.7k Ω (DHT pull-up), 1k Ω (Buzzer)	Current limiting and signal conditioning

Power Supply	USB or 5V adapter	System power (9V can be used with regulator)
---------------------	-------------------	--

System Workflow:

7. Initialization: Setup serial communication, initialize sensor, and LCD
8. Data Collection: Read temperature from TMP36 every 2 seconds
9. Display: Show current readings on 16x2 LCD
10. Threshold Checking: Compare current temperature against set limits
11. Alert Triggering: Activate buzzer and LEDs if thresholds are exceeded

3.3 Circuit Connections

TMP36 Temperature Sensor:

- VCC → Arduino 5V
- GND → Arduino GND
- DATA → Arduino Analog Pin A0 ()

16x2 LCD Display (Parallel Interface):

- VCC → Arduino 5V
- GND → Arduino GND
- RS → Arduino Pin 12
- Enable → Arduino Pin 11

LED Indicators:

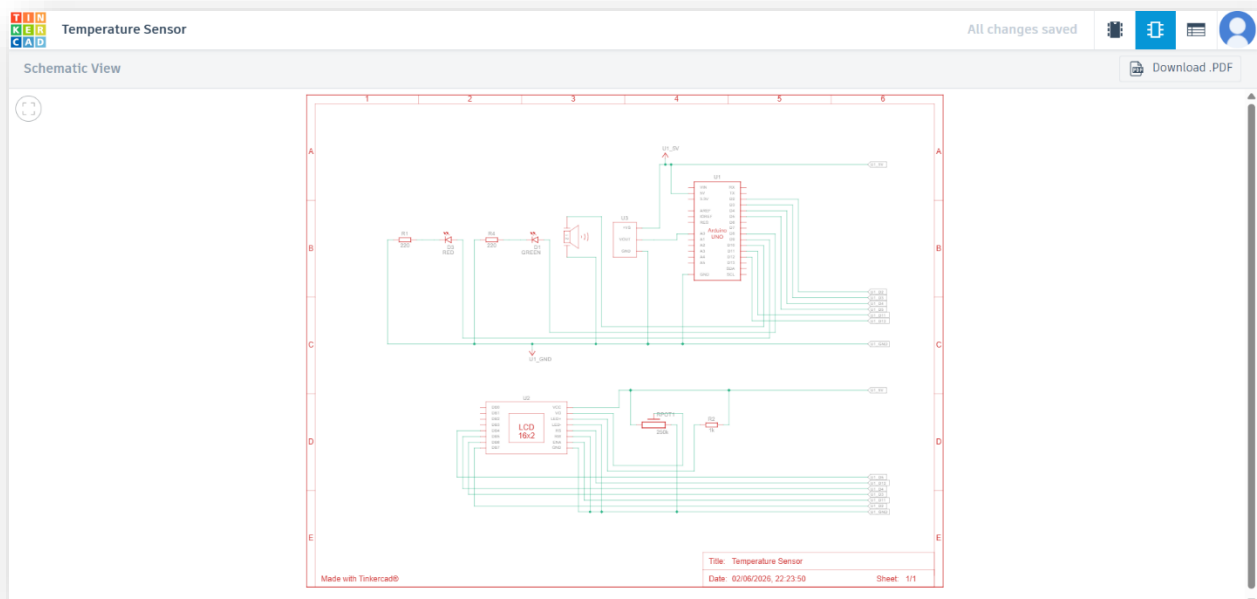
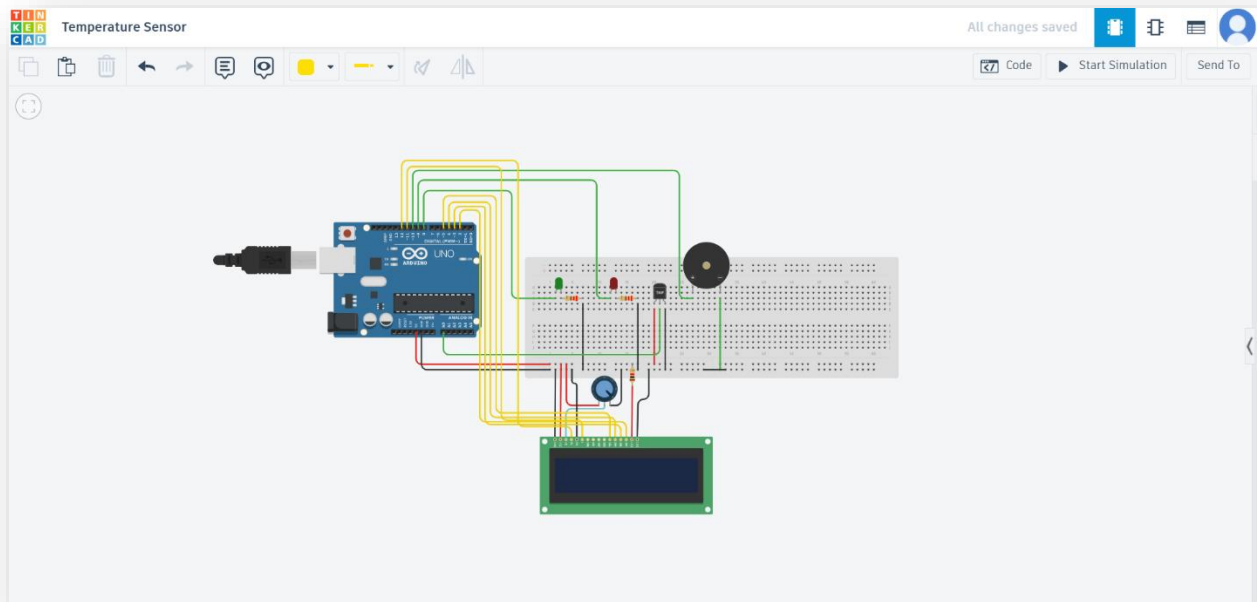
- Green LED (Normal) → Arduino Pin 8
- Red LED (Alert) → Arduino Pin 9
- Include 220Ω current-limiting resistor for each LED

Buzzer:

- Positive → Arduino Pin 10 (via 1kΩ resistor)
- Negative → Arduino GND

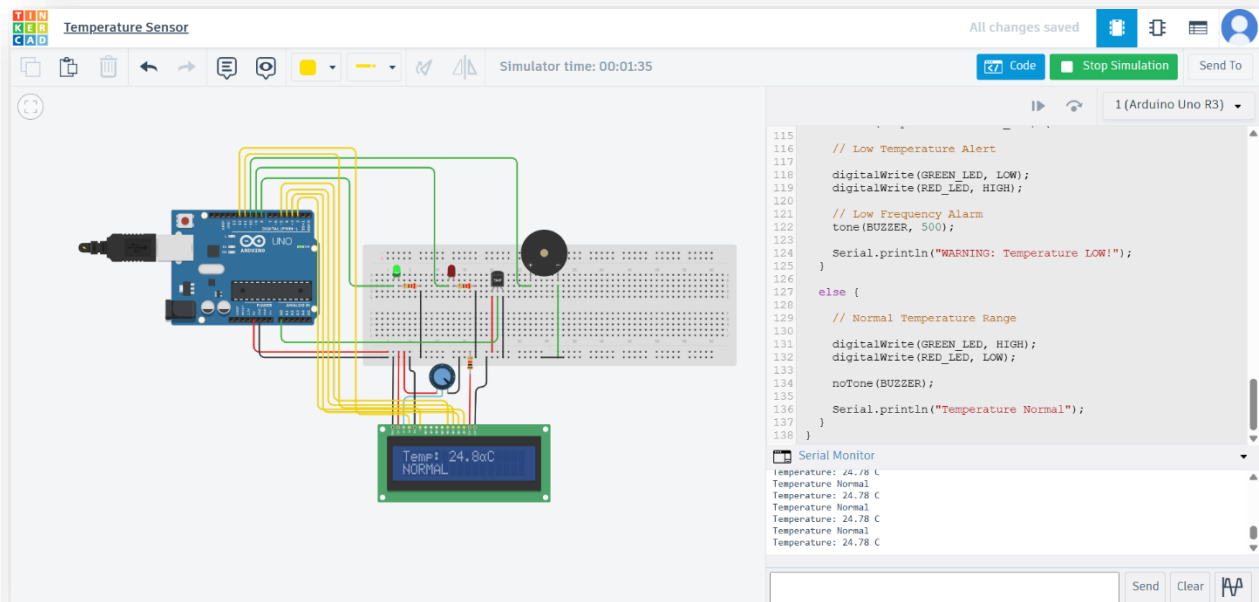
3.4 Simulation Results

1) Circuit Diagram

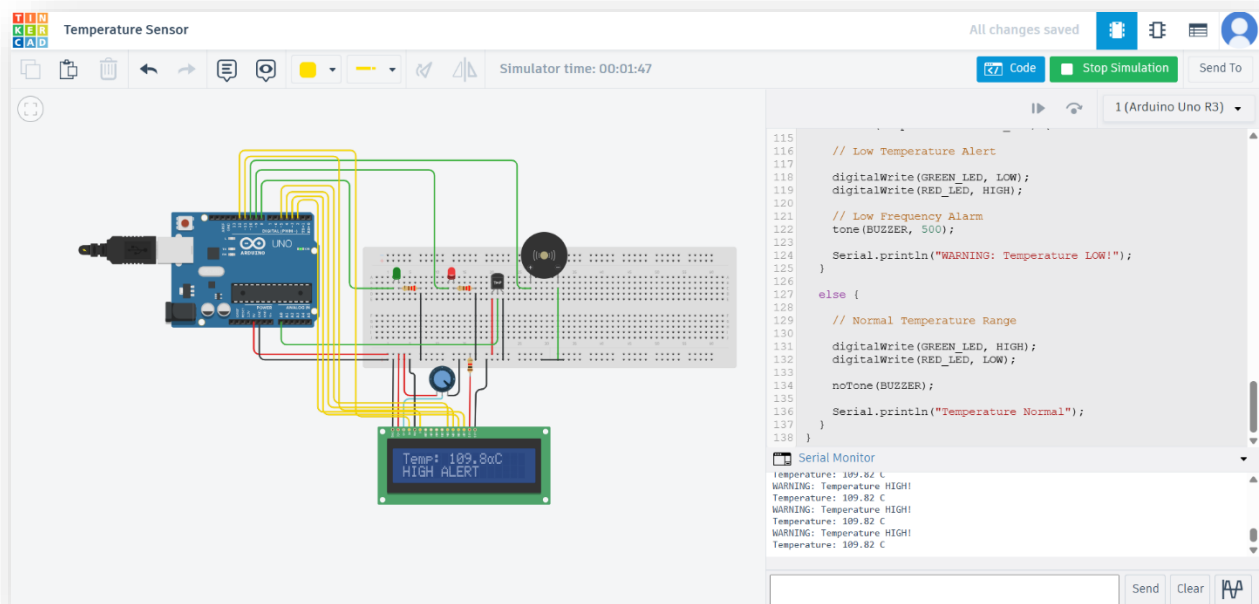


(Schematic Diagram)

2) Normal Temperature Condition



3) High Temperature Condition



4. Implementation & Code

4.1 Arduino C++ Code

Required Libraries:

#include <LiquidCrystal.h>

```
#include <LiquidCrystal.h> // LCD Library
```

Pin Definitions and Initialization:

```
// Pin Definitions
```

```
// LCD Pin Configuration
```

```
// RS, E, D4, D5, D6, D7
```

```
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
```

```
#define TEMP_SENSOR A0      // TMP36 output connected to Analog Pin A0
```

```
#define GREEN_LED 8         // Green LED indicates normal temperature
```

```
#define RED_LED 9           // Red LED indicates alert condition
```

```
#define BUZZER 10           // Buzzer for audio alerts
```

```
// Temperature Thresholds
```

```
const float TEMP_HIGH = 30.0;      // Upper temperature limit (°C)
```

```
const float TEMP_LOW = 15.0;       // Lower temperature limit (°C)
```

Setup Function:

```
void setup() {
```

```
// Initialize Serial Monitor
```

```
Serial.begin(9600);
```

```
// Initialize LCD (16 columns, 2 rows)
```

```
lcd.begin(16, 2);
```

```
// Configure Output Pins
```

```
pinMode(GREEN_LED, OUTPUT);
```

```
pinMode(RED_LED, OUTPUT);
```

```
pinMode(BUZZER, OUTPUT);
```

```
// Display Welcome Message
```

```
lcd.setCursor(0, 0);
```

```
lcd.print("TEMP MONITOR");
```

```
lcd.setCursor(0, 1);
```

```
lcd.print("Initializing");

delay(2000);

lcd.clear();

Serial.println("Smart Temperature Monitor Initialized");
}
```

Main Loop Function:

```
void loop() {

// Read analog value from TMP36 sensor
int sensorValue = analogRead(TEMP_SENSOR);

// Convert analog reading to voltage
float voltage = sensorValue * (5.0 / 1023.0);

// Convert voltage to temperature in Celsius
float temperature = (voltage - 0.5) * 100.0;

// Display temperature on LCD
displayReadings(temperature);

// Check thresholds and trigger alerts
checkThresholds(temperature);

// Print temperature to Serial Monitor
Serial.print("Temperature: ");
Serial.print(temperature);
Serial.println(" C");

delay(1000); // Update every 1 second
}
```

Display Function:

```
void displayReadings(float temperature) {

lcd.clear();

// First Row: Temperature Display
lcd.setCursor(0, 0);
lcd.print("Temp: ");
lcd.print(temperature, 1);
lcd.print((char)223); // Degree Symbol
lcd.print("C");

// Second Row: Status Message
lcd.setCursor(0, 1);
```



```
if (temperature > TEMP_HIGH) {  
  lcd.print("HIGH ALERT");  
}  
  
else if (temperature < TEMP_LOW) {  
  lcd.print("LOW ALERT");  
}  
else {  
  lcd.print("NORMAL");  
}  
}
```

Threshold Check Function:

```
void checkThresholds(float temperature) {  
  
  if (temperature > TEMP_HIGH) {  
  
    // High Temperature Alert  
  
    digitalWrite(GREEN_LED, LOW);  
    digitalWrite(RED_LED, HIGH);  
  
    // High Frequency Alarm  
    tone(BUZZER, 1000);  
  
    Serial.println("WARNING: Temperature HIGH!");  
  }  
  
  else if (temperature < TEMP_LOW) {  
  
    // Low Temperature Alert  
  
    digitalWrite(GREEN_LED, LOW);  
    digitalWrite(RED_LED, HIGH);  
  
    // Low Frequency Alarm  
    tone(BUZZER, 500);  
  
    Serial.println("WARNING: Temperature LOW!");  
  }  
  
  else {  
  
    // Normal Temperature Range  
  
    digitalWrite(GREEN_LED, HIGH);  
    digitalWrite(RED_LED, LOW);  
  
    noTone(BUZZER);  
  
    Serial.println("Temperature Normal");  
  }  
}
```

```
}  
}
```

4.2 Code Explanation

Key Components:

1. Threshold Logic

The `checkThresholds()` function implements a simple state machine. If temperature exceeds `TEMP_HIGH` or drops below `TEMP_LOW`, the red LED activates and buzzer produces an alert pattern. Otherwise, the green LED indicates normal operation.

2. Digital Output Control

`digitalWrite(pin, HIGH)` sets the pin to 5V, and `digitalWrite(pin, LOW)` sets it to 0V. Delays ensure users perceive the LED and buzzer changes. The buzzer pattern (short vs long beeps) helps users distinguish between different alert conditions.

5. Real-World Applications

5.1 Direct Applications of This Project

Cold Chain Monitoring

In pharmaceutical and food industries, temperature monitoring is critical. This system can ensure vaccines, medicines, and perishable goods remain within safe temperature ranges during storage and transport. Alert notifications enable immediate corrective action to prevent spoilage.

Server Room Monitoring

Data centers must maintain strict temperature control to prevent hardware failures. Multiple smart monitors throughout the facility provide real-time alerts when cooling systems malfunction, preventing costly equipment damage and data loss.

Greenhouse Automation

Agricultural IoT systems use temperature monitoring with automated ventilation and heating controls. Thresholds trigger actions like opening vents when too hot or activating heating when too cold, optimizing crop growth.

Industrial Incubators

Medical and biological incubators require precise temperature control. Smart monitors ensure samples remain at optimal conditions and alert staff immediately to deviations.

Home Climate Control

Smart homes integrate multiple temperature monitors for zone-based climate control. The system learns user preferences and automatically adjusts HVAC settings for comfort and energy efficiency.

5.2 Scalability & Future Enhancements

Cloud Integration

- Send temperature data to Firebase or AWS for cloud storage and analysis

- Access historical data and trends from anywhere via mobile app

Machine Learning

- Predict equipment failures using historical temperature patterns
- Anomaly detection to identify unusual temperature behavior

Multi-Sensor Integration

- Add pressure sensors, air quality sensors, or light sensors
- Create comprehensive environmental monitoring stations

Wireless Connectivity

- Upgrade to ESP32 for Wi-Fi/Bluetooth capabilities
- Enable remote monitoring and control via web dashboard

6. Conclusion

This project demonstrates fundamental embedded systems concepts including microcontroller programming, sensor integration, I2C communication, and real-time data processing. The Smart Temperature Monitor serves as a foundation for more complex IoT applications.

6.1 Learning Outcomes

- Proficiency in Arduino C++ programming and microcontroller operation
- Understanding of digital and analog sensor interfaces
- Experience with embedded systems design and troubleshooting
- Ability to design and implement real-world IoT solutions

6.2 Next Steps

12. Build and test the project on real hardware or Tinkercad
13. Add data logging to SD card module for long-term storage
14. Integrate with cloud platform for remote monitoring
15. Develop mobile app for smartphone alerts and control
16. Deploy in real-world application scenario

6.3 References & Resources

- Arduino Official Documentation: <https://www.arduino.cc/reference>
- Tinkercad Circuits: <https://www.tinkercad.com/circuits>
- ESP32 Documentation: <https://docs.espressif.com/>